# EXPERIMENT – 9

## Title: Implementation of Cyclic Redundancy Check (CRC)

---

## Objective:

To study and implement CRC technique for error detection in data communication.

---

## Software / Tools Required

- Operating System: Linux
- Programming Language: Python 3
- Custom Module: netpack

## Theory

Cyclic Redundancy Check (CRC) is an error detection technique used in computer networks and digital communication systems. It is mainly used to detect errors that may occur during data transmission.

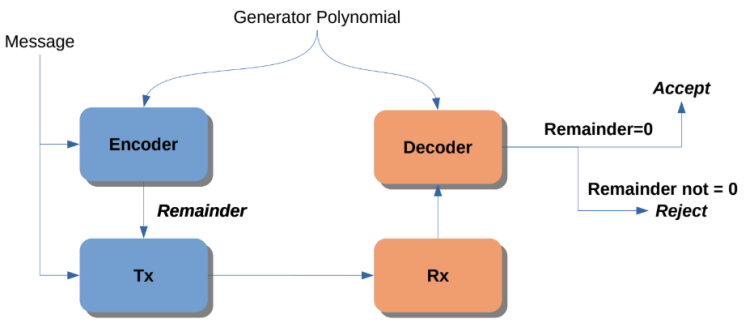
In CRC, the sender and receiver both use a generator polynomial. The sender divides the message bits by the generator polynomial using modulo-2 division. The remainder obtained after division is called CRC bits or remainder.

The sender appends this remainder to the original message and sends the coded word to the receiver. At the receiver side, the received data is again divided using the same generator polynomial.

If the remainder becomes zero, then the received data is accepted because no error is detected. If the remainder is non-zero, then the receiver assumes that an error has occurred during transmission and rejects the data.

CRC provides better error detection capability compared to simple parity checking methods and is widely used in networking protocols and data communication systems.

## Scheme of CRC:



**Procedure / Algorithm**

1. Define message data bits.
2. Define generator polynomial.
3. Perform modulo-2 division.
4. Generate CRC remainder.
5. Append remainder with original data.
6. Send coded word to receiver.
7. Receiver again performs CRC checking.
8. If remainder is zero, accept data.
9. Otherwise reject the received data

Program Code:

```
import netpack as npk

data = '100100'
key = '1101'
error_flag=False

print("[=] Message string : {}".format(data))
print("[=] Generator polynomial : {}".format(key))

c = npk.CRC_Class()

rm,CW=c.encode_data(data, key)
print("[=] Remainder is : {}".format(rm))
print("[=] Sender coded word is : {}".format(CW))

if error_flag:
    # Introduce error:
    val=list(CW)
    if val[-1]=='0':
        val[-1]='1'
    else:
        val[-1]='0'
    CW=''.join(val)
    print("-"*50)
    print("Introducing error in sender data")
    print("-"*50)

print("[=] Code word to receiver : {}".format(CW))

rm, msg=c.decode_data(CW, key)
print("[=] Receiver side remainder : {}".format(rm))
print("[=] Received message has [{}]".format(msg))
```

Output:

```
sumit@VivoBook-16X:/mnt/c/Users/sumit/Documents/B TECH 3rd YEAR STUFF/6th sem/Networking/exp-9$ python3.10 9_crc.py
[=] Message string : 100100
[=] Generator polynomial : 1101
-----------------------------------------------
[=] CRC object is created
-----------------------------------------------
[=] Remainder is : 001
[=] Sender coded word is : 100100001
[=] Code word to receiver : 100100001
[=] Receiver side remainder : 000
[=] Received message has [No Error]
sumit@VivoBook-16X:/mnt/c/Users/sumit/Documents/B TECH 3rd YEAR STUFF/6th sem/Networking/exp-9$
```

**Result:**

The CRC error detection technique was implemented successfully using Python.

---

**Conclusion:**

The experiment helped to understand CRC generation, modulo-2 division, and error detection during data transmission.